DEVELOPER20003 · OPEN-SOURCE / SOURCE-AVAILABLE IOT STACK

Offline Linux IoT Platform Beginner's Customization & Development Guide

Understand the system. Change display languages, SMS providers, dashboards, device behavior and application code. Build, test and publish changes safely on openSUSE.

Platform: openSUSE Leap 16.0 · x86_64

Baseline: EMQX Enterprise 6.3.1 · Kafka 4.2.2 · ThingsBoard CE 4.3.1.6 · Zigbee2MQTT 2.14.2 · PostgreSQL 17

Version: 1.0 · 10 November 2026

Audience: Beginners and maintainers of the two Developer20003 GitHub repositories

Publication status: This is an instructional guide, not certification that every source component can already be rebuilt offline. Each unverified operation is labeled and must be tested in an isolated copy first.

READING MAP

Contents

Begin with the overview, then choose the hands-on tutorial that matches your goal. Commands marked **candidate** are not yet verified offline builds.

01	Start here: what you have, and what this guide will teach
02	1. Understand how the whole IoT system works
03	2. Your openSUSE server: folders, images and commands
04	3. Choose the safest customization level
05	4. Tutorial A: customize display language in ThingsBoard CE
06	5. Tutorial B: configure SMS without changing Java source
07	6. Tutorial C: add a new SMS provider safely
08	7. Customizing EMQX and Erlang/Elixir
09	8. Customizing Kafka, the Java bridge, Zigbee2MQTT and BLE
10	9. Exact offline build workflow: safe laboratory procedure
11	10. A complete worked example: add multilingual SMS alarms
12	11. Version control, patches and GitHub publication
13	12. How to troubleshoot common failures
14	13. Roadmap: what “fully supported offline” will actually mean
15	Appendix A. First-week learning path
16	Appendix B. Glossary for non-specialists
17	Appendix C. Verified sources and further reading

Evidence basis: the published Developer20003 repositories, your openSUSE terminal audits, pinned ThingsBoard source, and linked official project documentation. Version: 10 November 2026.

Start here: what you have, and what this guide will teach

You already have the building blocks, but not a fully proven offline development release. The original repository publishes source and runtime bundles, dependencies and Docker images; the new repository publishes Linux development documentation, scripts and customization placeholders. Your openSUSE server has the four main Git source trees, cached dependencies, builder images and running EMQX/Kafka/ThingsBoard/PostgreSQL containers. Previous project notes report an MQTT-to-Kafka-to-ThingsBoard bridge test. However, complete, clean offline source rebuilds of EMQX, Kafka, ThingsBoard and Zigbee2MQTT are **not yet verified**.

This guide explains the practical difference between **configuration**, **customization** and **source development**. Start with configuration whenever possible. Build a changed application only when configuration cannot meet the requirement.

If you want to…	Start here	Need source rebuild?
Change a user's available ThingsBoard interface language	User profile language setting	No
Correct or add ThingsBoard CE translation text	`ui-ngx/src/assets/locale/`	Usually yes for a packaged customized UI
Configure Twilio, AWS SNS or SMPP SMS	ThingsBoard system SMS settings, if present in the installed CE UI	Normally no
Use a new SMS company or local GSM modem	Separate SMS adapter service / supported local gateway	No core rebuild if integrated externally
Implement a provider **inside** ThingsBoard	Java SMS sender interfaces, configuration types, UI and tests	Yes
Change MQTT topics, rules or Kafka mapping	EMQX rules/bridges and the Java bridge	Usually no core rebuild
Change broker authentication protocol	EMQX configuration first; Erlang source if necessary	Maybe
Change Kafka itself	Kafka Java/Scala source	Yes, and significant testing
Customize dashboards and alert rules	ThingsBoard administrator UI	Usually no
Add Zigbee device behavior	Zigbee2MQTT configuration, converters or TypeScript source	Maybe

Important vocabulary: "Offline build" means the compiler cannot fetch packages from the public internet. "Offline runtime" may still use the private LAN. **Real SMS delivery** requires a reachable carrier/service, such as an internet SMS API, a private SMPP network, or a cellular modem with network coverage. An internet-disconnected computer cannot magically transmit SMS without a communication path.

The two GitHub repositories

Repository	Purpose	Main content
`Developer20003/iot-offline-source-kit`	Baseline distribution, originally assembled around Windows/WSL and Docker	Seven large release ZIPs, source Git bundles, Docker image archives, Maven/Gradle/package caches, compiled artifacts and restoration scripts
`Developer20003/iot-offline-linux-devkit`	Native Linux companion development repository	Version manifests, Linux audit scripts, bridge build/import scripts, Compose template, docs and patch directories

The Linux repository currently contains **tools and instructions, not a second copy of the complete Erlang/Kafka/ThingsBoard toolchains**. Large assets are referenced from the original release where possible. Do not duplicate these multi-gigabyte archives in ordinary Git commits.

Exact pinned source versions

Source	Version	Git commit (prefix)	Principal implementation language
EMQX Enterprise	6.3.1	`b060492619`	Erlang and Elixir; native dependencies
Apache Kafka	4.2.2	`c5e9428595`	Java and Scala
ThingsBoard Community Edition	4.3.1.6	`c77335f4a5`	Java backend, Angular/ TypeScript frontend
Zigbee2MQTT	2.14.2	`344b6e03`	TypeScript / Node.js
Custom Kafka → ThingsBoard bridge	Local authored component	Not yet part of initial GitHub source commit	Java

> **Safe default:** Do not update upstream versions while learning. The cached dependencies and pinned Docker images were collected for these exact revisions. Updating a minor version can invalidate the offline cache.

1. Understand how the whole IoT system works

1.1 The data path, in plain language

```
Zigbee sensors / BLE devices / MQTT devices
        |                 |
        | Zigbee / BLE    | MQTT
        v                 v
Zigbee2MQTT / BLE-to-MQTT gateway
        |
        | MQTT topic + JSON telemetry
        v
      EMQX 6.3.1  ---- rules / Kafka sink ----> Apache Kafka 4.2.2
                                                      |
                                                      | Topic: iot-telemetry
                                                      v
                                        Java Kafka-to-ThingsBoard bridge
                                                      |
                                                      | ThingsBoard HTTP telemetry API
                                                      v
                                         ThingsBoard CE 4.3.1.6
                                                      |
                                                      v
                                             PostgreSQL 17
                                                      |
                                                      v
                                       Dashboards / alarms / users
                                                      |
                           (optional) SMS alert / email / external adapter
```

Zigbee2MQTT talks to Zigbee hardware and publishes device data via MQTT. **EMQX** accepts and routes MQTT messages; rules can transform or forward the messages. **Kafka** stores a stream of events for consumers. **The Java bridge** reads Kafka records, maps each device to its ThingsBoard identity, and sends telemetry to ThingsBoard. **ThingsBoard** stores, displays and alerts on device data. **PostgreSQL** retains the application database. An **SMS provider** carries an outgoing alert to a phone.

1.2 Example: temperature alarm

A sensor reports `{"device":"warehouse-1","temperature":38.2}`. Zigbee2MQTT (if the sensor is Zigbee) sends it to an MQTT topic. EMQX receives it and its Kafka sink forwards it to `iot-telemetry`. The Java bridge maps `warehouse-1` to the correct ThingsBoard device token and POSTs telemetry. ThingsBoard displays the value and may create an alarm when a defined rule is met. An alarm rule or external integration may send a message such as "Warehouse 1 temperature is high."

The thresholds, access rights and message routing are configuration or business logic. You generally do **not** need to rewrite the Kafka broker or EMQX core to add an alarm.

1.3 What is separate from the message path?

- **User interface language**: translations seen on ThingsBoard pages; these do not change sensor data or SMS transport.
- **Programming language**: Erlang/Elixir, Java, TypeScript and Python used to implement the services; you normally extend each service in its existing language.

- **SMS message language**: the text of an outgoing message; can be chosen from a template without translating the entire user interface.
- **Device language/protocol**: MQTT, Zigbee, BLE, HTTP and Kafka are protocols, not display languages.

2. Your openSUSE server: folders, images and commands

2.1 Where files are located

The locations below are specific to your current Linux server, not generic paths from upstream instructions.

```
# The restored working source/runtime kit:
/home/admin/projects/iot-offline-development

# The GitHub Linux companion repository:
/home/admin/projects/iot-offline-linux-devkit

# Disposable experiments and test builds:
/home/admin/projects/iot-offline-build-sandbox
```

Important subdirectories:

Location relative to `iot-offline-development/`	Meaning
`sources/emqx/`	EMQX editable source code
`sources/kafka/`	Kafka editable source code
`sources/thingsboard/`	ThingsBoard editable source code
`sources/zigbee2mqtt/`	Zigbee2MQTT editable source code
`caches/emqx/`	Available EMQX offline dependency cache
`caches/gradle/`	Kafka Gradle cache resources
`caches/maven/`	ThingsBoard Maven dependencies
`caches/yarn/` , `caches/pnpm/`	JavaScript dependency resources
`runtime/`	Running application files and settings - **protect**
`gateway/offline-kafka-thingsboard/`	Local Java bridge source and working files, when present

Current Linux repository location: `~/projects/iot-offline-linux-devkit`. Some published instructions still contain an older `-starter` path; substitute the actual directory.

2.2 The Docker images already present

Your audit showed these exact local images:

```
postgres:17
eclipse-temurin:17-jdk
maven:3.9.9-eclipse-temurin-17
node:22.22.2-bookworm
python:3.12
ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04
```

A Docker image is a prepackaged tool environment. You may not have `javac`, `mvn`, `node` or `gcc` installed on the openSUSE **host**, but the correct tool may already exist **inside a builder image**. Host command absence is not, by itself, a missing development capability.

2.3 First commands to learn (read-only)

Run one block at a time. Nothing below starts, stops or rebuilds services.

```
# 1. Go to the Linux development repository.
cd ~/projects/iot-offline-linux-devkit

# 2. Confirm you are tracking the published repository.
git status

# You should see branch main and no unexpected changes.
```

```
# Check the real runtime without changing it.
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'

# Display locally stored builder images.
docker images

# Inspect the pinned source revisions.
cat ~/projects/iot-offline-linux-devkit/versions/source-commits.txt

# Run the existing published audit script.
bash ~/projects/iot-offline-linux-devkit/scripts/audit-offline-dev.sh
```

Explanation: `cd` changes your shell folder. `git status` shows pending local changes. `docker ps` lists running containers. `docker images` lists available images. `cat` reads a text file. `bash script.sh` runs a shell script. `--network none` in a build command blocks container networking. `--pull never` prevents Docker from downloading an image.

2.4 Critical safety rules

1. **Never edit files under live runtime volumes** just to test new code. Do not run database schema installation on an existing ThingsBoard database.
2. **Never use `docker compose down -v`** on this system: the `-v` option can remove named volumes, including persistent data.
3. Keep EMQX node data, Kafka KRaft `meta.properties`, PostgreSQL data and ThingsBoard credentials intact.

4. Test in a **disposable working copy**, not directly on `sources/emqx`, `sources/kafka`, or the live ThingsBoard JAR.
5. Do not commit `.env`, access tokens, database dumps, keys or customer phone numbers to GitHub.
6. Build with `--network none` and `--pull never`. Do not "fix" an offline build failure by temporarily enabling internet and still call the test offline.
7. Some Docker bind mount flags such as `:z` change SELinux labels on host files. Use them only on disposable directories, not on the original baseline or secret directories unless you understand the consequences.
8. Take verified backups before making deployment or data migrations; keep the backup outside GitHub.

3. Choose the safest customization level

Level 1: Use an existing setting. Example: update an EMQX rule, a ThingsBoard alarm threshold or an existing SMS provider. This is the preferred approach.

Level 2: Customize your own bridge or adapter. Example: translate SMS text, map a device to a tenant, add an SMS provider endpoint, or introduce retries. This is safer than modifying a large upstream application and usually easier to test offline.

Level 3: Modify an upstream application's source. Example: add a new ThingsBoard locale, modify a core SMS provider factory, change EMQX Erlang authentication logic or extend Kafka's broker implementation. This requires complete source builds, tests and a new deployed artifact.

Requirement	Suggested owner	Why
Company logo or dashboard layout	ThingsBoard dashboard/UI	Avoids changing Kafka/EMQX
Spanish or French interface text	ThingsBoard user locale / UI locale files	Translation belongs to frontend
Alerts in Indonesian/Arabic	ThingsBoard notification template or SMS adapter	SMS text need not match UI locale
Local GSM modem SMS	Dedicated SMS adapter	Isolates modem, serial ports and provider errors
New cloud SMS API	Dedicated SMS adapter	Avoids deep changes to ThingsBoard
Custom device-to-tenant mapping	Java Kafka bridge	Bridges source and destination identity
MQTT authentication or client policies	EMQX config/rules, then Erlang code if needed	Broker owns MQTT access
Telemetry retention	ThingsBoard/PostgreSQL/Kafka configuration	Better than rewriting storage engine

4. Tutorial A: customize display language in ThingsBoard CE

4.1 First try the built-in language selection (no programming)

Open the ThingsBoard UI from a browser on the host that can reach your local deployment. Your prior Compose maps the web service to `127.0.0.1:8080` on the **openSUSE machine**. `localhost` on Windows refers to Windows; if openSUSE runs in a VMware VM, network forwarding or a secure VM-to-host route may be needed. Do not expose port 8080 publicly just to view it.

1. Sign in as an authorized ThingsBoard user.
2. Open your user account/profile menu.
3. Find the **Language** setting and choose an available language.
4. Save, then reload the page if necessary.
5. Check menus, dashboards, dates, units, alarms and form messages.

This changes the user's display language. It does not translate sensor values already stored in PostgreSQL.

> **Community Edition warning:** ThingsBoard online documentation also describes a **White Labeling → Custom Translation** administration feature. That custom-translation API/UI is listed as **Professional Edition-only** in the official REST API reference. Do not assume your **ThingsBoard CE 4.3.1.6** installation has that feature. The reliable CE development route is to inspect and modify its frontend locale source and build a customized UI.

4.2 Find the exact ThingsBoard translation files

The source tree at your pinned commit contains files such as:

```
sources/thingsboard/ui-ngx/src/assets/locale/
    locale.constant-en_US.json
    locale.constant-es_ES.json
    locale.constant-fr_FR.json
    locale.constant-de_DE.json
    locale.constant-zh_CN.json
    ...
```

List the language files **without editing anything**:

```
cd ~/projects/iot-offline-development/sources/thingsboard
find ui-ngx/src/assets/locale -maxdepth 1 -name 'locale.constant-*.json' \
  -printf '%f\n' | sort
```

`en_US` is U.S. English; `es_ES` is Spain Spanish; `fr_FR` is French. Do not confuse a **locale filename** with a programming-language file extension. The `.json` holds key/value translations.

4.3 Copy the source to a safe test folder

This can consume around 2 GB or more. Your previous audit showed 41 GB free, so check again first.

```
df -h "$HOME"
mkdir -p "$HOME/projects/iot-offline-build-sandbox/language-test"

# Copy only when the destination does not exist.
SRC="$HOME/projects/iot-offline-development/sources/thingsboard"
DST="$HOME/projects/iot-offline-build-sandbox/language-test/thingsboard"
if [ -e "$DST" ]; then
  echo "STOP: test destination already exists; inspect it first."
else
  cp -a --reflink=auto "$SRC" "$DST"
fi
```

This is a **copy operation**, not a rebuild. It does not modify the original ThingsBoard source or the running ThingsBoard container. Keep unrelated test changes out of the copy.

4.4 Change a translation safely

Open this copied file in an editor:

```
~/projects/iot-offline-build-sandbox/language-test/thingsboard/
  ui-ngx/src/assets/locale/locale.constant-es_ES.json
```

Example of a JSON section:

```
{
  "account": {
    "account": "Cuenta",
    "notification-settings": "Configuración de notificaciones"
  }
}
```

This is an **illustration**, not a replacement for the complete JSON file. In your real file, change **only the translated value** on the right. Never change the translation key such as `"notification-settings"`. Also preserve placeholders such as `{{value}}`, format markers, HTML tags, punctuation constraints and Unicode encoding.

Validate the JSON syntax immediately:

```
F="$HOME/projects/iot-offline-build-sandbox/language-test/thingsboard/ui-ngx/src/assets/locale/
locale.constant-es_ES.json"
python3 -m json.tool "$F" >/dev/null && echo 'TRANSLATION JSON: VALID'
```

Compare against the original:

```
diff -u \
  "$HOME/projects/iot-offline-development/sources/thingsboard/ui-ngx/src/assets/locale/
locale.constant-es_ES.json" \
  "$F" || true
```

The diff should show only the intended translated strings. If it shows thousands of reformatting changes, revert the formatting and use a simpler text editor.

4.5 Adding a language that does not exist

Adding Indonesian, for example, is **more than copying a JSON file**: the application may need a locale registry entry, language selection label, loader configuration and packaging rules. Do not invent a new locale file and assume the UI will automatically list it.

A safe discovery process is:

```
cd ~/projects/iot-offline-build-sandbox/language-test/thingsboard

grep -R -n 'locale.constant-es_ES\|es_ES' \
  ui-ngx/src/app ui-ngx/src/assets 2>/dev/null | head -50
```

Locate the code that registers the available locales, add the new locale consistently, validate all translation keys against English, and test both existing and new language selection. A right-to-left language also needs direction/layout testing. **Do not modify the production database to add a frontend language.**

4.6 Build and publish the changed UI

A modified JSON file is **source**, not a deployed update. ThingsBoard CE's backend/Angular frontend build is Maven-based and uses Node frontend dependencies. Your project has a Maven builder image, a cached Maven repository, and `ui-ngx/node_modules`, but a complete network-disabled rebuild is **not yet proven**.

Do not replace the running `thingsboard-4.3.1.6-boot.jar` based on an untested build. Use the isolated build strategy in Section 9. Only after the UI build and regression tests pass should you package a new artifact and follow a rollback-controlled deployment.

5. Tutorial B: configure SMS without changing Java source

5.1 Understand the difference between an SMS provider and an SMS alert

An **SMS provider** is the transport: it knows how to send a message to a phone. An **SMS alert** is the business rule: it decides *when* to send and *what the text says*. Configure these separately. Otherwise, switching providers may unexpectedly change alarm behavior.

At the exact ThingsBoard 4.3.1.6 source revision, the Java SMS sender factory includes **AWS SNS**, **Twilio**, and **SMPP**. Your installation's UI and role permissions determine whether they can be configured through the administration page.

Provider	Network requirement	Beginner use case
Twilio	Usually public internet to Twilio APIs	Quick cloud-based alerting
Amazon SNS	Usually public internet to AWS endpoints	AWS-integrated deployments
SMPP	Connection to an SMSC/provider; can be private LAN if supplied	Telecom/private SMS gateway

Provider	Network requirement	Beginner use case
Local GSM modem adapter	Local USB/serial access plus cellular network	Site with no public internet but available mobile coverage
Local mock adapter	No external SMS connection	Safe offline tests; sends **no real SMS**

Offline constraint: A Docker application on a private LAN can still send messages through a reachable local SMS gateway. A fully isolated `--network none` test container cannot reach even a private SMPP server unless test traffic is explicitly injected in-process. Cloud SMS API delivery is not "fully offline."

5.2 Check for the built-in ThingsBoard SMS setting

1. Log in with an authorized **System Administrator** account.
2. Navigate to **Settings → Notifications**, if available in your installed version.
3. Look for **SMS provider type**.
4. Choose **Twilio**, **AWS SNS**, or **SMPP**, if offered.
5. Obtain provider credentials through its official management portal; never put them in GitHub.
6. Save the configuration only after verifying sender identity, permissions and network access.
7. Use the application's **Send test SMS** feature with an authorized phone number, if available.
8. Inspect application logs for sending errors, masking secrets and phone numbers before sharing any output.

If the settings page or provider choice is absent in your exact CE build, stop and inspect your deployed version and source. Do not assume UI parity with Professional Edition or with newer documentation.

5.3 Example: Twilio settings

A Twilio configuration typically needs **Account SID**, **authentication token**, and a **sender phone number or messaging service identifier**. Recipients should use **E.164** formatting, e.g. `+12025550123` (an illustrative format only). Save credentials in the application/private configuration and test against the actual provider. Internet access and an active Twilio account are necessary for real delivery.

5.4 Example: private SMPP settings

An SMPP configuration typically needs: **host, TCP port, system ID, password, SMPP version**, and optionally bind type, source address and TON/NPI. Your pinned Java implementation supports SMPP version 3.3 or 3.4. Before production, ask the provider for its binding parameters, messaging limits, character encoding, delivery receipts and network allowlist requirements.

If your SMS provider is on a private LAN, test that the ThingsBoard container can reach **that specific host and port** without granting general internet access. Do not open a public inbound port solely for testing.

5.5 Configure *when* an SMS is sent

A common approach is to create a ThingsBoard device alarm rule or rule chain: incoming temperature → threshold comparison → alarm creation → notification/template → SMS. The exact menu labels depend on your CE 4.3.1.6 UI and selected role.

Example policy:

```
Device: warehouse-1
Trigger: temperature > 35 C for 5 minutes
Severity: MAJOR
Recipient group: on-call-maintenance
Message: "Alert: Warehouse 1 temperature exceeded 35 C."
Cooldown: one SMS per alarm transition; prevent repeated sends
```

The purpose of a cooldown is to avoid an SMS flood if the sensor sends a new message every few seconds. Configure notification policies, alert de-duplication, acknowledgement and recovery behavior before connecting to a billable SMS gateway.

5.6 Translate the outgoing SMS text

You do **not** need to translate the whole ThingsBoard application to send multilingual alerts. Keep separate message templates, for example:

Key	English	Spanish
`alarm.high_temp`	`High temperature at {device}: {value} C`	`Temperatura alta en {device}: {value} C`
`alarm.recovered`	`Temperature normal at {device}`	`Temperatura normal en {device}`

Choose the message language based on the **recipient's preference**. Keep placeholders stable; apply correct Unicode/SMS encoding and account for longer multi-segment messages and provider charges. Never translate device IDs or JSON field names accidentally.

6. Tutorial C: add a new SMS provider safely

6.1 Recommended: an external provider adapter

If you need a local telecommunications company, custom REST SMS API or GSM modem, the **least disruptive architecture** is:

```
ThingsBoard alarm or notification event
            |
            | Internal REST / message request
            v
  Dedicated SMS Adapter (your code)
            |
       Provider selection
        /     |       \
   Twilio   SMPP     GSM modem / custom API
            |
            v
      Delivery result / audit
```

This isolates provider-specific code. It also lets you add a second provider, fail over between providers, implement SMS language templates and test with a mock, without rebuilding ThingsBoard's large Java application.

Possible ThingsBoard integration: use a supported **REST API call rule node**, notification webhook or a separate consumer of a dedicated Kafka topic (subject to what your installed CE rule chain exposes). A REST node is one established mechanism in current official ThingsBoard CE documentation. Test availability in your exact build before relying on it.

6.2 Define a small provider-neutral API

Design an internal endpoint for your adapter. This is a **proposed interface**, not an existing service in your repositories:

```
POST /api/notify/sms HTTP/1.1
Content-Type: application/json
Authorization: Bearer <INTERNAL_SECRET_NOT_IN_GIT>
```

```
{
  "event_id": "alarm-warehouse-1-20261110T090000Z",
  "device_id": "warehouse-1",
  "recipient_ref": "on-call-maintenance",
  "locale": "en_US",
  "template": "alarm.high_temp",
  "parameters": {"value": 38.2, "unit": "C"}
}
```

Security: This contract deliberately uses `recipient_ref` rather than an arbitrary phone number supplied by a device. The adapter resolves phone numbers from an administrator-controlled contact directory. Authenticate the request, authorize allowed templates/recipients, apply rate limits, and protect private phone data.

Suggested response semantics:

- `202 Accepted`: request safely queued; **not** proof of SMS delivery.
- `400 Bad Request`: invalid event or template.
- `401/403`: missing authorization or forbidden recipient.
- `429`: rate limit or alert storm.
- `503`: provider unavailable; retry with backoff and a bounded queue.

6.3 Required adapter internals

```
request validation
   -> authenticate and authorize
   -> resolve recipient and language
   -> deduplicate by event_id + recipient + template
   -> select provider and render text
   -> queue and attempt submission
   -> store provider request ID/status
   -> retry transient failures
   -> persist final outcome / audit
```

Use a retry policy that distinguishes network timeouts from permanent invalid-recipient errors. Do not repeatedly send the same alarm each time Kafka replays a message. Protect logs: a provider token, the complete SMS text and personal phone numbers should not appear in public GitHub issues.

6.4 Select an implementation language

Need	Reasonable choice
Reuse existing Kafka bridge integration	Java 17
Build a separate HTTP/SMPP/GSM adapter rapidly	Python 3.12
MQTT policy/authentication deep inside the broker	Erlang/Elixir (EMQX)
ThingsBoard internal Java provider and settings	Java 17, plus TypeScript frontend

Your system does **not** need to be rewritten into one programming language. The services already communicate using standard network protocols.

6.5 If you insist on adding the provider inside ThingsBoard

The **pinned source paths below were checked against ThingsBoard commit `c77335f4a5`**:

```
application/src/main/java/org/thingsboard/server/service/sms/
  AbstractSmsSender.java
  DefaultSmsSenderFactory.java
  DefaultSmsService.java
  twilio/TwilioSmsSender.java
  smpp/SmppSmsSender.java
  aws/... (AWS sender)
```

The existing `DefaultSmsSenderFactory` selects `AWS_SNS`, `TWILIO` or `SMPP`. Adding another option normally requires **more than one new Java class**:

1. Create a new `SmsSender` implementation, potentially extending `AbstractSmsSender` for helper validation.
2. Extend the configuration type/data model and its serialization/deserialization.
3. Update the provider type enumeration and factory dispatch.
4. Add required provider configuration fields to the administration UI and validation paths.
5. Update security/redaction of credentials, dependency injection, testing and user-facing errors.
6. Provide compatibility handling for existing stored settings, then rebuild and test the entire relevant app.
7. Do not introduce public cloud SDK dependencies unless their exact binaries are staged for offline compilation.

Do not edit `DefaultSmsSenderFactory.java` in the live restored source and expect the running JAR to change. The running ThingsBoard process uses compiled code. Source changes need a build and controlled deployment.

7. Customizing EMQX and Erlang/Elixir

7.1 What you can change without Erlang coding

Start in the EMQX Dashboard: MQTT listeners, client authentication, authorization policies, rules, connectors and bridges. Your prior configuration used a rule `offline_mqtt_to_kafka`, connector `offline_kafka_connector` and sink `offline_telemetry_sink` to forward to Kafka topic `iot-telemetry`.

Examples of **configuration-level** changes:

- Change topic routing from `sensors/+/temperature` to an appropriate new topic filter.
- Add a new bridge sink to Kafka for another device category.
- Set client permissions so each device may publish only its approved topic.
- Change rule transformations and field names, with regression tests.

These operations can affect live traffic. Export configuration and test in a separate instance where possible.

7.2 When Erlang/Elixir source is actually needed

EMQX source is under `sources/emqx/`. Modules are organized under `apps/`; for example, the pinned tree has `apps/emqx_bridge_kafka`, `apps/emqx_auth_http`, and `apps/emqx` directories. Modify these only for behavior that configuration cannot express.

The builder image contains the **EMQX fork of Erlang/OTP**, Elixir, Rust and build tools. This matters: EMQX upstream explicitly states that its forked OTP is required for supported source builds. Installing generic Erlang from the openSUSE package repository is **not equivalent**.

Pinned toolchains from your Linux manifest:

```
EMQX_ERLANG_FORK_OTP=28.4.1-4
EMQX_ELIXIR=1.19.1-otp-28
EMQX_RUST=1.88.0
EMQX_BUILDER_IMAGE=ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04
```

7.3 The known EMQX offline build gap

Earlier isolated tests confirmed Hex and Rebar3, 125 dependency directories and previously compiled BEAM files. However, the disposable `mix deps.check` could not validate dependencies because the root lock state was missing, and `mix deps.get` attempted to access an upstream Quicer Git repository. You inspected an existing local Quicer checkout and tag; this is a **dependency resolution/lock issue**, not proof that Quicer source is absent.

Status: not yet a reproducible offline clean build. Do not describe it as complete in the GitHub README until an isolated modified-source compile succeeds with networking disabled.

7.4 First Erlang learning exercise (read-only)

```
cd ~/projects/iot-offline-development/sources/emqx

# Look for Kafka bridge modules.
find apps/emqx_bridge_kafka -maxdepth 3 -type f | head -30

# Inspect the pinned toolchain version.
cat .tool-versions

# Check source cleanliness.
git status --short
```

For a first modification, consider changing a **simple rule/transform** or adding a **unit test** rather than editing MQTT packet handling, cluster state, authentication internals or Kafka delivery semantics.

8. Customizing Kafka, the Java bridge, Zigbee2MQTT and BLE

8.1 Kafka is a message broker, not your SMS provider

Kafka's job is to transport durable event streams. The `iot-telemetry` topic is read by the Java bridge. You usually change **topic configuration or consumer application logic**, not Kafka's broker source, to support SMS notifications or new telemetry schemas.

Kafka developer stack: Java 17 + Scala 2.13 + Gradle wrapper + pinned offline caches. Upstream Kafka 4.2.2 documents `./gradlew jar` and Gradle unit/integration test tasks. Gradle still needs a cached wrapper distribution, plugins and artifacts even with `--offline` enabled.

A useful low-risk customization is to update your Java bridge with:

1. A device/token map loaded from a **private** configuration file.
2. Schema validation for incoming JSON.
3. A bounded retry queue for ThingsBoard HTTP failures.
4. Kafka manual offset commits only after the chosen delivery acknowledgement policy is satisfied.
5. Idempotency/deduplication strategy when replaying records.
6. Unit tests for device mapping and multilingual alert generation.

8.2 Your Java bridge is not yet fully in the new GitHub repo

The current Linux repository tracks `gateway/offline-kafka-thingsboard/src/.gitkeep`, not the authored Java implementation itself. It does have an import script:

```
cd ~/projects/iot-offline-linux-devkit
bash scripts/import-local-bridge.sh
```

This **copies** a Java source file from the working Linux kit if present and refuses to overwrite an existing destination. Before committing it, inspect the Java file for passwords, device tokens, private URLs or embedded phone numbers. The published `scripts/build-offline-bridge.sh` builds the imported source using the local Kafka distribution libraries and JDK 17 image with `--network none --pull never`.

8.3 Zigbee2MQTT

The Zigbee2MQTT source is under `sources/zigbee2mqtt/`. The project is written in TypeScript; upstream documents `pnpm run build` after dependencies are installed. The last Linux audit found a small `dist/` directory but **no** `node_modules/` in the restored Zigbee2MQTT source tree.

Beginner path:

1. Inspect `package.json` to see available scripts and exact dependency tools.
2. Change the **configuration** for a device first, not TypeScript source.
3. Back up the Zigbee2MQTT `data/configuration.yaml` and device database if you later enable the hardware service.

4. Verify that the pnpm store and an appropriate pnpm executable can be used **offline** with your Node builder image.
5. Build and test in an isolated copy; **do not pair/reset devices** as part of a source-build test.

8.4 BLE-to-MQTT gateway

The original kit includes Python 3.12 and a Python dependency cache. For a BLE gateway, changes usually belong in device scan logic, data parsing, MQTT topics, reconnection policies and device permission handling. On real Linux, USB and Bluetooth interface permissions differ from Windows/WSL. Hardware tests are still pending; do not claim a complete BLE deployment until a real adapter and sensor work after reboot.

9. Exact offline build workflow: safe laboratory procedure

9.1 Do not confuse runtime success with compilation success

- **Runtime PASS:** An existing JAR, BEAM release or Kafka distribution starts and processes messages.
- **Source compile PASS:** Your edited source produces a new artifact locally.
- **Offline compile PASS:** The edited source compiles with `--network none`, `--pull never`, and no uncached external downloads.
- **Reproducibility PASS:** The same pinned inputs build again from a separate clean/disposable environment.
- **Deployment PASS:** A test instance runs the changed artifact, passes regression tests and can roll back.

Do not move an item from one column to the next without a retained test log and artifact checksum.

9.2 Make a disposable source copy before building

Choose **one component** per experiment. Example for Kafka:

```
ROOT="$HOME/projects/iot-offline-development"
TEST="$HOME/projects/iot-offline-build-sandbox/kafka-local-test"

# Safety preflight: stop if this directory already exists.
if [ -e "$TEST" ]; then
  echo "STOP: test directory already exists; do not overwrite it."
else
  mkdir -p "$TEST"
  cp -a --reflink=auto "$ROOT/sources/kafka" "$TEST/kafka"
  cp -a --reflink=auto "$ROOT/caches/gradle" "$TEST/gradle-home"
  echo "Copies created; original source and caches preserved."
fi
```

The copied `gradle-home` is **writable** for Gradle in the test environment. Caches must be staged with the expected Gradle home layout; a cache directory merely existing does not prove the wrapper and plugins are complete.

9.3 Kafka: candidate offline build test

Only run after the disposable copy exists and after confirming the cache layout and Docker permissions. This is a **candidate test**, not a previously passed command.

```
TEST="$HOME/projects/iot-offline-build-sandbox/kafka-local-test"

docker run --rm --network none --pull never \
  --user "$(id -u):$(id -g)" \
  --workdir /kafka \
  --volume "$TEST/kafka:/kafka:z" \
  --volume "$TEST/gradle-home:/gradle-home:z" \
  --env GRADLE_USER_HOME=/gradle-home \
  --entrypoint /bin/bash eclipse-temurin:17-jdk \
  -lc './gradlew --offline --no-daemon jar'
```

If Gradle cannot find its distribution, it may still try to fetch that distribution; `--network none` makes the attempt fail. Record the exact missing version/artifact and stage it from a connected packaging computer. Never change the **running Kafka data directory** or reinitialize KRaft to fix a compile error.

9.4 ThingsBoard: candidate offline Maven build test

Make a separate ThingsBoard **source copy** as in Section 4.3. Copy `caches/maven/` to a writable test cache; do not mount the shared cache writable from multiple builds.

```
ROOT="$HOME/projects/iot-offline-development"
TEST="$HOME/projects/iot-offline-build-sandbox/language-test"

# Precondition: $TEST/thingsboard exists and contains the source copy.
if [ ! -d "$TEST/maven-cache" ]; then
  cp -a --reflink=auto "$ROOT/caches/maven" "$TEST/maven-cache"
fi

docker run --rm --network none --pull never \
  --user "$(id -u):$(id -g)" \
  --workdir /thingsboard \
  --volume "$TEST/thingsboard:/thingsboard:z" \
  --volume "$TEST/maven-cache:/offline-maven:z" \
  --entrypoint /bin/bash maven:3.9.9-eclipse-temurin-17 \
  -lc 'mvn -o -Dmaven.repo.local=/offline-maven -DskipTests package'
```

Status: unverified. This may fail on a missing Maven plugin, frontend Yarn/Node dependency, build profile or packaging requirement. The example does not guarantee a full frontend build. Keep the exact error and fix the offline resource bundle rather than silently accessing the internet.

9.5 EMQX: candidate isolated compilation

Use the pre-existing disposable EMQX sandbox or create a **new** copy; avoid changing `sources/emqx/` directly. The correct EMQX builder image must be used. Illustrative command, **not yet a passing build**:

```
TEST="$HOME/projects/iot-offline-build-sandbox/emqx-local-test"
ROOT="$HOME/projects/iot-offline-development"

if [ -e "$TEST" ]; then
  echo "STOP: choose an unused test directory."
else
  cp -a --reflink=auto "$ROOT/sources/emqx" "$TEST"
fi

# Only after the copy is verified and EMQX offline Hex/Git caches
# are appropriately mounted or staged in the disposable test:
docker run --rm --network none --pull never \
  --user "$(id -u):$(id -g)" \
  --workdir /emqx \
  --volume "$TEST:/emqx:z" \
  --entrypoint /bin/bash \
  ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04 \
  -lc 'make emqx-enterprise-compile'
```

The `mix` dependency lock issue described in Section 7.3 must still be resolved. This candidate may fail; do not claim success based on a previously compiled `_build/` tree.

9.6 Zigbee2MQTT: offline prerequisites before build

The Node image was present, but `pnpm` activation and complete dependency restoration have **not been proven**. Start with a read-only inspection:

```
cd ~/projects/iot-offline-development/sources/zigbee2mqtt
cat package.json | head -75

docker run --rm --network none --pull never \
  --entrypoint /bin/sh node:22.22.2-bookworm \
  -c 'node --version; npm --version; command -v corepack || true; command -v pnpm || true'
```

Then prepare a **disposable Zigbee2MQTT source copy** plus a writable copy of the pnpm cache. Only after the pinned pnpm executable and store are validated should you run the source's own `pnpm run build` in a network-disabled container. A missing package is a packaging task, not permission to use a live internet download in the "offline" test.

9.7 What to capture for every build

Evidence	Why it matters
Source commit hash and `git diff`	Exact source provenance
Docker image name + immutable digest	Toolchain provenance
Cache manifest SHA256 values	Dependency integrity
Exact full command and environment	Reproducibility
`--network none` / `--pull never` test evidence	No downloads occurred

Evidence	Why it matters
Compilation log and exit code	Real result, not assumption
SHA256 of generated artifact	Artifact identity
Test result, deployment result and rollback note	Operational safety

Do not upload logs that reveal credentials or personal information.

10. A complete worked example: add multilingual SMS alarms

This is a **recommended implementation plan**, not an assertion that the example has already been deployed.

Goal

A temperature sensor reports a high temperature. ThingsBoard creates an alarm. English-speaking staff receive an English alert; Spanish-speaking staff receive a Spanish alert. A custom local provider can later be swapped without editing the alarm logic.

Design decisions

1. **Sensor schema:** `device_id`, `temperature`, `timestamp` are validated by your bridge; device credentials stay in private configuration.
2. **Alarm rule:** threshold and debounce belong in ThingsBoard or a dedicated rules component, not in Kafka's broker code.
3. **Recipient directory:** `on-call-maintenance` resolves to approved contacts, time zone, preferred language and an opt-in/permission policy.
4. **SMS adapter:** accepts an internal authenticated request; chooses template and provider.
5. **Provider selection:** choose Twilio/SMPP/GSM based on available connectivity; local mock for offline tests.
6. **Deduplication:** one notification per `alarm_id + recipient_ref + channel`; retries do not resend completed messages.
7. **Evidence:** record acceptance, provider submission ID, final delivery status if the provider supports it, and errors without storing secrets in logs.

Test sequence

Step	Expected output	Pass condition
1. Publish one test MQTT message	EMQX receives known JSON	Message visible in permitted test trace
2. Check Kafka `iot-telemetry`	Record contains expected device	Consumer receives the message
3. Check Java bridge	Correct destination device	ThingsBoard receives telemetry

Step	Expected output	Pass condition
4. Trigger test alarm	One alarm instance	No alert storm
5. Use mock SMS provider	English or Spanish rendered text	Correct locale and placeholders
6. Repeat the same event ID	Duplicate suppressed	No second SMS request
7. Simulate provider outage	Retry/queue with status	No data loss or infinite loop
8. Configure real approved provider	Test SMS to authorized phone	External delivery confirmed
9. Restart test services	State recovers	No duplicate storm
10. Restore baseline artifact	Original path works	Rollback verified

Do not send real test messages to third-party phone numbers without their permission. Real SMS can incur charges and require regulatory compliance.

11. Version control, patches and GitHub publication

11.1 Separate upstream source from your own modifications

Keep upstream source and third-party dependencies in the restored kit; keep **your authored changes** in the Linux companion repository as scripts, patches, tests, documentation and permitted release assets.

```
iot-offline-linux-devkit/
  docs/
    IOT_Offline_Linux_Customization_Guide.pdf
    IOT_Offline_Linux_Customization_Guide.md
  scripts/
  versions/
  patches/
    emqx/
    kafka/
    thingsboard/
    zigbee2mqtt/
  gateway/
    offline-kafka-thingsboard/
```

The PDF is for readers. Keep the Markdown beside it for searchable diffs and future updates; the PDF should be regenerated when the Markdown changes.

11.2 How to maintain a custom source change

For one upstream project, start from the pinned commit in an isolated working copy. Make one focused change, test it, commit it locally and export a patch. Example *after you have made and tested a ThingsBoard source commit*:

```
# Use the project copy where YOU made the change, not the original live tree.
CUSTOM="$HOME/projects/iot-offline-build-sandbox/language-test/thingsboard"
KIT="$HOME/projects/iot-offline-linux-devkit"

git -C "$CUSTOM" status --short
# Commit your change within CUSTOM after review (not shown).

# Export commits after the pinned baseline SHA as patch files.
git -C "$CUSTOM" format-patch \
  c77335f4a524bb9fe0209806672c6ec7aea76141 \
  -o "$KIT/patches/thingsboard"
```

Only publish original work and patches you are permitted to redistribute. Review diffs for embedded passwords, phone numbers, network information and license headers.

11.3 Add this guide to your GitHub repository

First download the PDF (and optional Markdown source) from this conversation, then copy them into your Linux repo's `docs/` directory. For example, **after files have been transferred to the Linux machine**:

```
cd ~/projects/iot-offline-linux-devkit

cp /path/to/IOT_Offline_Linux_Customization_Guide.pdf docs/
cp /path/to/IOT_Offline_Linux_Customization_Guide.md docs/

# Preview exactly what is changing.
git status --short
git diff --check

# Stage only the two guide files.
git add docs/IOT_Offline_Linux_Customization_Guide.pdf \
        docs/IOT_Offline_Linux_Customization_Guide.md

git diff --cached --stat

# After reviewing the files:
git commit -m 'Add beginner guide for offline Linux IoT customization'
git push origin main
```

Note: `/path/to/...` must be replaced by the actual directory in which you saved the downloaded files. The GitHub token is entered at the interactive HTTPS prompt if requested; do not embed it in a command line or README.

11.4 New release assets, not giant Git commits

Use GitHub Release attachments for verified large cache archives and builder image tarballs, with SHA256 manifests. Record a link to the **original** release `v0.1.0-offline-preview`, rather than uploading identical gigabytes again. Keep license notices for third-party components. EMQX Enterprise 6.3.1 is source-available under BSL 1.1; check the exact redistribution/commercial requirements before distributing modifications or binaries.

12. How to troubleshoot common failures

Symptom	Likely explanation	Safe next action
`mvn: command not found` on openSUSE	Maven exists inside Docker builder, not host	Use `maven:3.9.9-eclipse-temurin-17`
`javac: command not found` on host	JDK builder image is separate	Run in `eclipse-temurin:17-jdk`
Gradle wants to download wrapper distribution	Cached Gradle distribution missing/ wrong home	Audit local cache; stage on connected packaging machine
Maven `Could not resolve...` in `-o`	Required artifact/plugin absent from local repo	Record precise coordinate and stage it
EMQX Mix requests GitHub Quicer	Missing/incorrect Git dependency resolution/lock	Fix isolated local mirror/lock handling; keep network blocked
Zigbee2MQTT `pnpm` unavailable	Node image does not imply pinned pnpm executable ready	Audit Corepack and staged package-manager assets
Translation JSON won't load	Invalid JSON, missing key or locale registry	Run JSON validator; compare keys; inspect frontend loader
SMS test fails but telemetry works	Provider credentials, recipient, provider route or network	Test provider separately, without exposing secrets
`localhost:8080` works in VM, not Windows	Loopback belongs to the VM	Use approved host networking or SSH tunnel
Docker container starts but data missing	Wrong volume, database state, device mapping or installer run	Stop changes; inspect mounts; restore from verified backup
Git push asks for password	HTTPS push requires Personal Access Token	Paste PAT at password prompt, not your account password
`src refspec main does not match any`	No local commit yet	Commit after staging and review

Never troubleshoot by doing these things

- Deleting `postgres` volumes or Kafka `meta.properties`.
- Running an installer on an already initialized ThingsBoard database.
- Removing Maven/Gradle caches from the only offline copy.
- Exposing EMQX dashboard, Kafka, PostgreSQL or private SMS adapter ports to the public internet.
- Publishing the Java bridge while it contains device tokens or hardcoded SMS credentials.
- Claiming an offline build passed solely because a prebuilt service runs.

13. Roadmap: what “fully supported offline” will actually mean

Release 1: Linux developer starter (the current state)

- ⊡ New Linux companion repository published.
- ⊡ Pinned versions recorded and basic audit/bridge scripts published.
- ⊡ Existing restored source trees and Docker images observed on openSUSE.
- ⊡ Existing runtime containers observed running.
- ☐ Import and review the authored Java bridge source into the companion repo.
- ☐ Verify source builds of each application with no network and changed code.

Release 2: reproducible offline source development

- ☐ EMQX Enterprise 6.3.1 customized-source build succeeds from a disposable copy.
- ☐ Kafka 4.2.2 customized-source Gradle build succeeds with `--offline`.
- ☐ ThingsBoard CE 4.3.1.6 backend/UI build succeeds with Maven `-o` and offline frontend packages.
- ☐ Zigbee2MQTT 2.14.2 packages restore, build and test fully offline.
- ☐ SMS mock adapter and multilingual notification tests pass.
- ☐ Add exact missing packages, cache-manifest hashes and build scripts to the release.

Release 3: deployable and recoverable platform

- ☐ Fresh disconnected openSUSE machine restores the images, caches, binaries and source.
- ☐ PostgreSQL, Kafka and EMQX persistent data survive a controlled reboot.
- ☐ Changed-source deployments have tests, health checks, rollback and documented artifacts.
- ☐ Actual Zigbee/BLE hardware and (if chosen) approved SMS provider are tested.
- ☐ Security and third-party license audit is complete.

The success criterion is concrete: take the released offline assets to a disconnected Linux machine, modify one line of meaningful application code, compile with no internet, run tests, deploy a changed artifact and roll back safely. Until then, publish the toolkit as a **preview/development-in-progress**, not as a certified fully offline rebuild platform.

Appendix A. First-week learning path

Day	Beginner exercise	Evidence to keep
1	Learn `cd`, `ls`, `cat`, `git status`, `docker ps`	Screenshot/text of read-only audit
2	Trace one known MQTT message from EMQX to Kafka	Topic and test payload (no credentials)
3	View one ThingsBoard device and its latest telemetry	Device mapping and sample values
4	Change the current user's display language; compare locale JSON	Before/after screenshots; JSON validation
5	Configure a test alarm without sending SMS	Alarm creation and acknowledgement
6	Run a mock SMS alert and test English/Spanish templates	Deduplication and alert mock log
7	Make one change in a disposable source copy and export a patch	Patch, test log and artifact checksum

Appendix B. Glossary for non-specialists

API: A defined software interface between services. **Artifact:** A file produced by a build, such as a JAR or EMQX release. **BEAM:** Erlang virtual-machine bytecode file. **Bridge:** Code that transfers data between systems. **Cache:** Downloaded dependencies preserved so builds can work offline. **CE:** Community Edition. **Commit:** A named snapshot in Git. **Container:** An isolated process filesystem and runtime launched from a Docker image. **E.164:** International phone-number format, such as `+12025550123`. **Erlang/Elixir:** Languages used by EMQX. **Frontend:** The web interface presented to users. **Gradle:** Kafka build tool. **i18n:** Internationalization: translation keys and locale support. **JAR:** Packaged Java application. **Kafka topic:** A named stream of events. **Maven:** ThingsBoard's Java build tool. **MQTT:** Messaging protocol frequently used by IoT devices. **OTP:** Erlang runtime and libraries. **PAT:** GitHub Personal Access Token for HTTPS Git authentication. **pnpm:** JavaScript package manager. **SMPP:** Telecommunications protocol for SMS gateways. **Source:** Human-readable code from which an application is built. **Telemetry:** Device measurements or readings. **VM:** Virtual machine.

Appendix C. Verified sources and further reading

Sources in this section were checked for the **specific repository content** or relevant public technical guidance. Features described in modern vendor docs may differ from the installed pinned version; the guide marks those cases clearly.

1. Original Developer20003 offline source kit and release assets v0.1.0-offline-preview - original distribution and dependency archives.
2. Developer20003 Linux companion devkit - README, manifests, scripts, and current limitations.
3. ThingsBoard CE translation source at pinned revision - confirmed locale JSON files.
4. ThingsBoard CE SMS sender factory at pinned revision - AWS SNS, Twilio and SMPP dispatch.
5. ThingsBoard CE SMS sender service at pinned revision - system SMS configuration and sender lifecycle.
6. Official ThingsBoard SMS provider settings - provider configuration and test UI; verify installed version and role.
7. Official ThingsBoard SMS rule node and external REST rule nodes - available integration designs.
8. Official ThingsBoard REST API reference - white-labeling/custom translation endpoints are marked PE-only.
9. Official ThingsBoard source build guide - Maven and Java requirements.
10. Official EMQX source build guidance - builder image and forked Erlang/OTP requirements.
11. Apache Kafka 4.2.2 source README - Java, Scala, Gradle build/test commands.
12. Zigbee2MQTT development instructions and configuration guide - TypeScript build and device settings.

Maintainer note: This document is designed for publication in `iot-offline-linux-devkit/docs/`. Update the Markdown and regenerate the PDF when pinned components, test evidence, paths, or SMS provider capabilities change. Do not publish private secrets or client data.